



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Numerical Methods for PDEs

High-Order Matrix-Free Solvers for
Advection-Diffusion-Reaction problems

Authors: **Luca Di Profio**
Simone De Carlo
Davide Cutrupi
Davide Di Tanna

Version: 1.0
Academic Year: 2025/2026

Contents

Contents	i
----------	---

1	Introduction	1
2	Mathematical Model	2
2.1	The Advection-Diffusion-Reaction problem	2
2.2	Weak Formulation	3
2.3	Galerkin Formulation	4
3	Stabilization: the SUPG Method	5
3.1	Galerkin instability and the local Péclet number	5
3.2	The SUPG formulation	5
3.3	Strong consistency and the residual simplification	6
3.4	The stabilization parameter τ_K	7
3.5	Interaction with the Chebyshev smoother	7
4	Numerical Methodology	8
4.1	Matrix-Based vs. Matrix-Free Approaches	8
4.2	Sum-Factorization: The Computational Engine	10
5	Precondition strategies	11
5.1	Geometric Multigrid (GMG) Preconditioner	11
5.1.1	Mesh Hierarchy and Finite Element Spaces	11
5.2	The V-Cycle Algorithm	12
5.3	Chebyshev Polynomial Smoothing	12
5.3.1	General Formulation and Symmetric Case	13
5.3.2	The Non-Symmetric Challenge and Complex Ellipses	13
6	Numerical Results	15
6.1	Convergence Analysis	15
6.1.1	Homogeneous Dirichlet boundary conditions	15
6.1.2	Inhomogeneous boundary conditions	16
6.2	Performance and Scalability Analysis	18
6.2.1	Pure diffusion–reaction	18
6.2.2	Advection-dominated transport (SUPG)	18
6.3	Strong Scaling	19

6.4	Weak Scaling	20
6.5	Polynomial Scaling	21
6.6	Memory Profiling	23
7	Conclusions	25
8	Bibliography	26

1 | Introduction

The computational efficiency of Finite Element Method (FEM) solvers on modern hardware is increasingly dictated by memory bandwidth rather than raw floating-point performance. As the polynomial degree p of the basis functions increases, traditional matrix-based methods suffer from a “memory wall”, where the CPU spends the majority of its time waiting for matrix entries to be transferred from the RAM.

This project explores the implementation of a **Matrix-Free** solver for the Advection-Diffusion-Reaction (ADR) equation. By re-computing the operator’s action locally on each cell instead of storing a global sparse matrix, we aim to transform the problem from being **memory-transfer bound** to **computation-bound**, leveraging vectorized CPU instructions and high parallel scalability.

2 | Mathematical Model

2.1 The Advection-Diffusion-Reaction problem

We consider a stationary Advection-Diffusion-Reaction (ADR) problem defined on a bounded domain $\Omega \subset \mathbb{R}^d$ ($d = 2, 3$), with a boundary $\partial\Omega = \Gamma_D \cup \Gamma_N$, where $\Gamma_D \cap \Gamma_N = \emptyset$. The strong form of the governing equation is given by:

$$\begin{cases} -\nabla \cdot (\mu \nabla u) + \nabla \cdot (\beta u) + \gamma u = f & \text{in } \Omega \\ u = g & \text{on } \Gamma_D \subset \partial\Omega \\ \nabla u \cdot \mathbf{n} = \psi & \text{on } \Gamma_N \subset \partial\Omega \end{cases} \quad (2.1)$$

The physical behavior of the system is characterized by the interaction of three distinct phenomena, each controlled by a specific coefficient:

- **Diffusion** (μ): The diffusion coefficient $\mu \in L^\infty(\Omega)$, represents the diffusivity of the medium. It dictates the rate at which the quantity u spreads isotropically.
- **Advection** (β): The vector field $\beta \in [C^1(\Omega)]^d$ represents the velocity of the medium. The term $\nabla \cdot (\beta u)$ models the transport of u along the streamlines of the flow.
- **Reaction** (γ): The scalar field $\gamma \in L^\infty(\Omega)$ describes local production or consumption processes. A positive coefficient ($\gamma > 0$) represents an absorption or "sink" term that dissipates the energy of the system, whereas a negative coefficient ($\gamma < 0$) acts as a "source" term.

2.2 Weak Formulation

Let $V = H^1(\Omega)$ and $V_0 = \{v \in V : v|_{\Gamma_D} = 0\}$. Multiplying the strong form (1) by a test function $v \in V_0$ and integrating over Ω , we obtain

$$-\int_{\Omega} \nabla \cdot (\mu \nabla u) v \, d\mathbf{x} + \int_{\Omega} \nabla \cdot (\beta u) v \, d\mathbf{x} + \int_{\Omega} \gamma u v \, d\mathbf{x} = \int_{\Omega} f v \, d\mathbf{x}, \quad \forall v \in V_0.$$

Integrating by parts the diffusive term through Green's first identity, a boundary integral over the whole $\partial\Omega$ appears:

$$-\int_{\Omega} \nabla \cdot (\mu \nabla u) v \, d\mathbf{x} = \int_{\Omega} \mu \nabla u \cdot \nabla v \, d\mathbf{x} - \int_{\partial\Omega} \mu (\nabla u \cdot \mathbf{n}) v \, ds.$$

Splitting the boundary as $\partial\Omega = \Gamma_D \cup \Gamma_N$, the boundary term reads

$$\int_{\partial\Omega} \mu (\nabla u \cdot \mathbf{n}) v \, ds = \int_{\Gamma_D} \mu (\nabla u \cdot \mathbf{n}) v \, ds + \int_{\Gamma_N} \mu (\nabla u \cdot \mathbf{n}) v \, ds.$$

Since $v|_{\Gamma_D} = 0$, the integral over Γ_D vanishes; on Γ_N we use the Neumann condition $\nabla u \cdot \mathbf{n} = \psi$, so that the boundary contribution reduces to $\int_{\Gamma_N} \mu \psi v \, ds$.

For the advective term,

$$\nabla \cdot (\beta u) = (\nabla \cdot \beta) u + \beta \cdot \nabla u$$

hence $\int_{\Omega} \nabla \cdot (\beta u) v \, d\mathbf{x} = \int_{\Omega} (\nabla \cdot \beta) u v \, d\mathbf{x} + \int_{\Omega} (\beta \cdot \nabla u) v \, d\mathbf{x}$.

To account for the non-homogeneous Dirichlet datum we use the lifting technique, writing $u = u_0 + R_g$ with $u_0 \in V_0$ and $R_g \in H^1(\Omega)$ such that $R_g|_{\Gamma_D} = g$. Collecting all the terms, the weak formulation reads: find $u_0 \in V_0$ such that

$$a(u_0, v) = F(v) - a(R_g, v) \quad \forall v \in V_0, \quad (2.2)$$

where

$$\begin{aligned} a(u, v) &= \int_{\Omega} \mu \nabla u \cdot \nabla v \, d\mathbf{x} + \int_{\Omega} (\nabla \cdot \beta) u v \, d\mathbf{x} + \int_{\Omega} (\beta \cdot \nabla u) v \, d\mathbf{x} + \int_{\Omega} \gamma u v \, d\mathbf{x}, \\ F(v) &= \int_{\Omega} f v \, d\mathbf{x} + \int_{\Gamma_N} \mu \psi v \, ds. \end{aligned} \quad (2.3)$$

2.3 Galerkin Formulation

Let $V = H^1(\Omega)$ and $\mathcal{T}_h$ be a partition (mesh) of Ω into N_{el} hexahedral elements K_c , $c = 1, \dots, N_{\text{el}}$, and let X_h^p be the space of piecewise tensor-product polynomials over the mesh elements,

$$X_h^p(\Omega) = \{v_h \in C^0(\overline{\Omega}) : v_h|_{K_c} \in \mathbb{Q}_r(K_c) \quad \forall c = 1, \dots, N_{\text{el}}\}.$$

Let $V_h = X_h^r \cap V$ be the finite element approximation of V , $V_{0,h} = X_h^p \cap V_0$, and $N_h = \dim(V_{0,h})$.

Restricting the weak formulation to $V_{0,h}$ and using a discrete lifting $R_{g,h}$, the Galerkin problem reads: find $u_{0,h} \in V_{0,h}$ such that

$$a(u_{0,h}, v_h) = F(v_h) - a(R_{g,h}, v_h) \quad \forall v_h \in V_{0,h}. \quad (2.4)$$

Let $\{\varphi_i\}_{i=1}^{N_h}$ be the Lagrangian basis of $V_{0,h}$. Looking for $u_{0,h} \in V_{0,h}$ in the form $u_{0,h} = \sum_{j=1}^{N_h} U_j \varphi_j$, and requiring the discrete problem to hold for each basis function φ_i , the Galerkin formulation rewrites as the linear system

$$A \mathbf{U} = \mathbf{F} \quad (2.5)$$

where

$$\begin{aligned} \mathbf{U} &\in \mathbb{R}^{N_h}, & \mathbf{U} &= (U_1, U_2, \dots, U_{N_h})^T, \\ A &\in \mathbb{R}^{N_h \times N_h}, & A_{ij} &= a(\varphi_j, \varphi_i) = \int_{\Omega} \mu \nabla \varphi_j \cdot \nabla \varphi_i \, d\mathbf{x} \\ & & &+ \int_{\Omega} (\gamma + \nabla \cdot \boldsymbol{\beta}) \varphi_j \varphi_i \, d\mathbf{x} \\ & & &+ \int_{\Omega} (\boldsymbol{\beta} \cdot \nabla \varphi_j) \varphi_i \, d\mathbf{x}, \\ \mathbf{F} &\in \mathbb{R}^{N_h}, & F_i &= F(\varphi_i) - a(R_{g,h}, \varphi_i) \\ & & &= \int_{\Omega} f \varphi_i \, d\mathbf{x} + \int_{\Gamma_N} \mu \psi \varphi_i \, ds - a(R_{g,h}, \varphi_i). \end{aligned} \quad (2.6)$$

Because of the advective terms $\nabla \cdot (\boldsymbol{\beta} u)$, the matrix A is non-symmetric.

3 | Stabilization: the SUPG Method

The standard Galerkin discretization of the advection–diffusion–reaction problem is unconditionally stable only when diffusion dominates. In the advection-dominated regime it loses control over the convective term and produces non-physical oscillations. To restore stability while preserving the accuracy of the high-order discretization, we employ the *Streamline-Upwind/Petrov–Galerkin* (SUPG) stabilization, a strongly consistent method that adds a residual-based, streamline-aligned perturbation to the variational formulation.

3.1 Galerkin instability and the local Péclet number

The balance between transport and diffusion on each element K_c of the mesh $\mathcal{T}_h$ is quantified by the local *Péclet number*

$$\text{Pe}_{K_c} = \frac{\|\beta\| h_{K_c}}{2\mu}, \quad (3.1)$$

where h_{K_c} is the local element size. When $\text{Pe}_{K_c} \lesssim 1$ the problem is locally diffusion-dominated and the Galerkin solution is stable. When $\text{Pe}_{K_c} \gg 1$ the element is too coarse to resolve the sharp boundary or internal layers induced by transport. The result is the appearance of spurious node-to-node oscillations that pollute the solution globally. Simply refining the mesh until $\text{Pe}_{K_c} \lesssim 1$ everywhere is prohibitively expensive in three dimensions, which motivates a stabilization acting at the discretization level.

3.2 The SUPG formulation

The test space differs from the trial space through a perturbation aligned with the streamlines,

$$v_h \longmapsto v_h + \tau_K (\beta \cdot \nabla v_h), \quad (3.2)$$

where $\tau_K > 0$ is an element-wise stabilization parameter defined in Section 3.4. The perturbation is applied to the strong residual of the equation

$$R(u) = -\nabla \cdot (\mu \nabla u) + \beta \cdot \nabla u + \gamma u - f, \quad (3.3)$$

so that the stabilized discrete problem reads: find $u_h \in V_h$ with $u_h = g_h$ on Γ_D such that, for all $v_h \in V_{0,h}$,

$$\begin{aligned} a(u_h, v_h) + \sum_{K \in \mathcal{T}_h} \int_K \tau_K (\boldsymbol{\beta} \cdot \nabla v_h) (-\nabla \cdot (\mu \nabla u_h) + \boldsymbol{\beta} \cdot \nabla u_h + \gamma u_h) d\mathbf{x} \\ = F(v_h) + \sum_{K \in \mathcal{T}_h} \int_K \tau_K (\boldsymbol{\beta} \cdot \nabla v_h) f d\mathbf{x}. \end{aligned} \quad (3.4)$$

Collecting the residual on a single side, the formulation can be written compactly as

$$a_h(u_h, v_h) = a(u_h, v_h) + \sum_{K \in \mathcal{T}_h} \int_K \tau_K (\boldsymbol{\beta} \cdot \nabla v_h) R(u_h) d\mathbf{x} = F(v_h), \quad (3.5)$$

which makes the structure of the method transparent: the stabilization adds extra diffusion only in the direction of the flow (through the factor $\boldsymbol{\beta} \cdot \nabla v_h$), and not in the transverse direction. This is where the oscillations arise, so the method removes them without blurring the solution, unlike a naive scheme that diffuses equally in all directions.

3.3 Strong consistency and the residual simplification

The decisive property of SUPG is *strong consistency*: the additional term in (3.5) is proportional to the residual $R(u_h)$, and the exact solution satisfies $R(u) = 0$ pointwise. Hence the exact solution still solves the stabilized weak form exactly, and the stabilization introduces no consistency error. This is what allows SUPG to remain accurate at high polynomial order.

In the present implementation the second-order term $-\mu \Delta u_h$ is omitted from the residual used inside the stabilization, which therefore reduces to

$$\tilde{R}(u_h) = \boldsymbol{\beta} \cdot \nabla u_h + \gamma u_h - f. \quad (3.6)$$

This is a standard simplification with two justifications. First, it avoids the evaluation of second derivatives of the shape functions at the quadrature points, which is convenient in the matrix-free setting; for affine $\mathbb{Q}_1$ elements the omitted term is moreover identically zero, since $\Delta u_h \equiv 0$ on each cell. Second, the stabilization is only active where advection dominates, i.e. precisely where the diffusive part of the residual is subdominant. We note, for completeness, that with the reduced residual (3.6) the exact solution gives $\tilde{R}(u) = \mu \Delta u \neq 0$ in general, so strict strong consistency is weakened to a consistency error of order $\mathcal{O}(\tau_K \mu \Delta u)$. In the advection-dominated regime where the term is switched on this contribution is negligible, and for the diffusion-dominated convergence study of Section 6.2.1 the stabilization is disabled altogether ($\tau_K = 0$), so the simplification has no effect on the reported error rates.

3.4 The stabilization parameter τ_K

The parameter τ_K must scale, balancing the advective, diffusive and reactive characteristic scales of the operator. We adopt the classical Shakib–Hughes/Codina form

$$\tau_K = \left[\left(\frac{2 \|\boldsymbol{\beta}\|}{h_K} \right)^2 + 9 \left(\frac{4\mu}{h_K^2} \right)^2 + \gamma^2 \right]^{-1/2}. \quad (3.7)$$

Each term corresponds to one regime: the first to advection ($\sim \|\boldsymbol{\beta}\|/h_K$), the second to diffusion ($\sim \mu/h_K^2$, with the high-order safety constant $9 \cdot 4^2$), and the third to reaction ($\sim \gamma$). The inverse-square combination ensures that τ_K is dominated by whichever scale is smallest (i.e. fastest), recovering the expected limits in each pure regime.

The local size h_K is recovered directly from the geometry available at each quadrature point, namely the inverse Jacobian of the reference-to-physical mapping: $h_K = 1/\max_i \|(J^{-1})_i\|$, the reciprocal of the largest row norm of J^{-1} , which measures the smallest geometric length scale of the cell.

3.5 Interaction with the Chebyshev smoother

Beyond accuracy, the stabilization has a beneficial effect on the preconditioner. As discussed in Section 5.3.2, the convective term makes the operator non-symmetric and pushes its eigenvalues into the complex plane, which is problematic for the real-valued Chebyshev smoother used in the multigrid V-cycle: outside its real optimization interval the Chebyshev polynomial grows exponentially and the iteration may diverge. By adding streamline diffusion, SUPG mitigates the skew-symmetric part of the advection operator and pulls the spectrum back towards the positive real axis. The smoother can then be applied safely on a real interval, with only a modest overestimation of $\lambda_{\max}$ as a safeguard, instead of having to track a family of complex confocal ellipses. The stabilization therefore plays a dual role: it improves both the accuracy of the discretization and the robustness of the preconditioner.

These results confirm that the implemented stabilization makes the solver robust in the transport-dominated regime, complementing the high-order accuracy verified in the diffusion-dominated convergence study.

4 | Numerical Methodology

4.1 Matrix-Based vs. Matrix-Free Approaches

The project compares two distinct computational strategies:

- **Matrix-Based:** The system matrix A is assembled and stored in a sparse format. For high-order elements, the number of non-zero entries per row grows as $O(p^d)$, leading to excessive memory consumption and slow data transfer.
- **Matrix-Free:** The matrix-free approach eliminates the memory bandwidth bottleneck by replacing matrix element lookups with local, on-the-fly evaluations. In practice, we compute the action of the linear operator on a given vector without explicitly assembling the global sparse matrix.

Let us start by looking at how the stiffness matrix A is assembled

$$A = \sum_{\text{cell}=1}^{\text{n_cells}} P_{\text{cell}}^T A_{\text{cell}} P_{\text{cell}} \quad (4.1)$$

where the matrix P_{cell} is a rectangular matrix that defines the index mapping from global DoFs to local ones in the current cell (P_{cell}^T viceversa), while A_{cell} denotes the cell matrix associated with A . Indeed:

$$A \cdot u = \left(\sum_{\text{cell}=1}^{\text{n_cells}} P_{\text{cell}}^T A_{\text{cell}} P_{\text{cell}} \right) \cdot u = \sum_{\text{cell}=1}^{\text{n_cells}} P_{\text{cell}}^T A_{\text{cell}} u_{\text{cell}} = \sum_{\text{cell}=1}^{\text{n_cells}} P_{\text{cell}}^T v_{\text{cell}}$$

where u_{cell} contains the values of u at the degrees of freedom of the respective cell and v_{cell} correspondingly for the result.

However, this approach is very slow. For every cell we generate a local matrix which takes three nested loops, leading to a complexity of $\mathcal{O}(\text{dofs_per_cell}^3)$. The multiplication itself is then done by two nested loops, which means that it is much cheaper.

To overcome this bottleneck, we can recognize that the local evaluation can be expressed as the application of three separate operators:

$$A_{\text{cell}} = B_{\text{cell}}^T D_{\text{cell}} B_{\text{cell}} \quad (4.2)$$

where:

- $\mathbf{B}$ maps the vector of local degrees of freedom (which are the coefficients of the basis functions) to the physical values or gradients at the quadrature points.
- $\mathbf{D}$ scales the values obtained by $\mathbf{B}$ at each quadrature point by the determinant of the Jacobian and the quadrature weights. Additionally it incorporates the coefficients describing the equation.
- $\mathbf{B}^T$ maps the weighted values from the quadrature points back to the local DoFs.

This decomposition avoids the explicit assembly of A_{cell} entries by evaluating the operator's action directly, thereby reducing the local computational complexity to $\mathcal{O}(\text{dofs_per_cell}^2)$.

However, the operators $\mathbf{B}$ and $\mathbf{D}$ are not constant across the mesh, as they must account for the geometric mapping to the physical space. In practice, values and gradients must be evaluated in the physical domain by transforming reference-cell quantities using the Jacobian of the mapping from the reference to the physical cell (e.g., the unit cube in 3D).

In a matrix-free implementation, these integrals are evaluated repeatedly during the iterative solution process. This necessitates a strategic choice regarding data caching: while pre-computing certain values saves CPU cycles, excessive caching leads to a memory-bandwidth bottleneck. Consequently, we do not store physical values and gradients explicitly within $\mathbf{B}$, as they vary for every basis function and quadrature point across the elements. Instead, we first apply the gradient on the reference cell only, factoring out the Jacobian transformation from the evaluation operator:

$$\mathbf{B}_{\text{cell}} = \begin{pmatrix} \mathbf{B}_{q_1} \\ \vdots \\ \mathbf{B}_{q_{N_q}} \end{pmatrix}, \quad \mathbf{B}_q = \begin{pmatrix} 1 & 0 \\ 0 & \mathbf{J}_q^{-T} \end{pmatrix} \hat{\mathbf{B}}_q, \quad (4.3)$$

where N_q denotes the total number of quadrature points in the cell.

In this decomposition, $\hat{\mathbf{B}}_q$ first interpolates the local DoF values to both values and reference-cell gradients at the quadrature points. The transformation is then applied selectively: the solution values remain in the reference space (as they are invariant under the mapping for H^1 elements), while the reference gradients are mapped to the physical space using the pre-cached inverse-transpose Jacobian $\mathbf{J}_q^{-T}$.

Once in the physical representation, the “physics kernel” $\mathbf{D}$ is applied. This operator is block-diagonal (the quadrature points do not couple) and encodes the PDE coefficients together with the integration weight:

$$\mathbf{D} = \text{diag}(\mathbf{D}_{q_1}, \dots, \mathbf{D}_{q_{N_q}}), \quad \mathbf{D}_q = w_q |\det(\mathbf{J}_q)| \begin{pmatrix} \gamma & \boldsymbol{\beta}^T \\ \mathbf{0} & \mu \mathbf{I}_{d \times d} \end{pmatrix}. \quad (4.4)$$

Acting on the pair (value, physical gradient) = $(u_q, \nabla u_q)^T$, this block produces $(\gamma u_q + \boldsymbol{\beta} \cdot \nabla u_q, \mu \nabla u_q)$.

Finally, the process is reversed for the integration step via the B^T operator: gradient contributions are mapped back to the reference space with $\mathbf{J}_q^{-1}$, while the value-based contributions are processed directly. This summation over quadrature points projects the physical interaction back onto the reference-cell test space, testing the first component against the test-function value v and the second against the test-function gradient ∇v . This operation exactly recovers the local weak form: $\int_K (\mu \nabla u \cdot \nabla v + (\boldsymbol{\beta} \cdot \nabla u) v + \gamma uv) \, d\mathbf{x}$.

4.2 Sum-Factorization: The Computational Engine

To avoid a complexity of $\mathcal{O}(p^{2d})$ operations per cell (the naive approach for a d -dimensional element), we utilize **Sum-Factorization**. By exploiting the tensor-product structure of hexahedral elements, the evaluation of the operator is decomposed into d one-dimensional steps. This reduces the complexity to $\mathcal{O}(d \cdot p^{d+1})$, making matrix-free methods significantly faster than matrix-based ones for $p > 2$.

The efficiency of this technique relies on the fact that the basis functions $\phi(\hat{\mathbf{x}})$ on a hexahedral reference cell can be represented as the product of one-dimensional Lagrange polynomials:

$$\phi_{i,j,k}(\hat{x}, \hat{y}, \hat{z}) = \varphi_i(\hat{x}) \varphi_j(\hat{y}) \varphi_k(\hat{z}) \quad (4.5)$$

where φ denotes the 1D basis polynomials. From an algebraic perspective, this analytical structure implies that the d -dimensional reference evaluation matrix $\hat{\mathbf{B}}$ can be expressed as the Kronecker product (tensor product) of small 1D evaluation matrices $\mathbf{B}_{1D}$:

$$\hat{\mathbf{B}} = \mathbf{B}_{1D} \otimes \mathbf{B}_{1D} \otimes \mathbf{B}_{1D} \quad (4.6)$$

When evaluating a function or its gradient at the quadrature points, this structure allows us to factor out the summations. Instead of performing a single d -dimensional interpolation, we perform a sequence of 1D operations along each coordinate axis. For instance, in 3D, the value at a quadrature point $(\hat{x}_q, \hat{y}_q, \hat{z}_q)$ is computed by first interpolating along the $\hat{x}$ -direction, then using those intermediate results to interpolate along $\hat{y}$ -direction, and finally along $\hat{z}$ -direction.

This strategy effectively replaces a dense matrix-vector product of size $(p+1)^d \times (p+1)^d$ with a series of smaller one-dimensional operations on blocks of size $(p+1)$. In a 3D simulation using polynomials of degree $p = 4$ (yielding 5 nodes per direction), this reduces the number of operations per cell from approximately $5^6 = 15,625$ to $3 \times 5^4 = 1,875$.

This polynomial reduction in complexity, lowering the exponent from $2d$ to $d+1$, combined with SIMD vectorization, ensures high performance even as the polynomial degree increases.

5 | Precondition strategies

5.1 Geometric Multigrid (GMG) Preconditioner

The fundamental principle of multigrid methods is to achieve uniform convergence rates by capturing and damping errors across multiple spatial resolutions. Traditional iterative solvers (smoothers) are highly effective at eliminating high-frequency, oscillatory components of the error but suffer from severe stall when dealing with low-frequency error components. The Geometric Multigrid (GMG) approach overcomes this limitation by transferring the low-frequency residual to a coarser mesh, where it appears relatively more oscillatory and can be efficiently damped.

5.1.1 Mesh Hierarchy and Finite Element Spaces

To formalize the GMG preconditioner, we define a hierarchy of quadrilateral (in 2D) or hexahedral (in 3D) meshes $\{\mathcal{T}_\ell\}$, for $0 \leq \ell \leq L$, obtained from a quasi-uniform coarse mesh $\mathcal{T}_0$ through consecutive refinement steps. The final hierarchy can be modeled as a tree graph oriented from coarse to fine cells, where the leaves constitute the active cells of the computational domain.

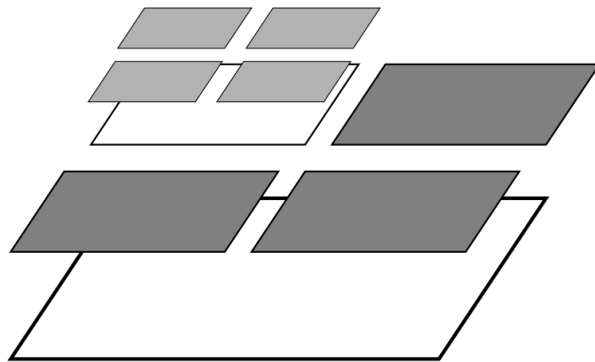


Figure 5.1: A hierarchy of three meshes with local adaptive refinement (active cells shaded).

Corresponding to this hierarchy, we construct a sequence of finite element spaces $V_\ell \subset H^1(\Omega)$, on the mesh $\mathcal{T}_\ell$. Denote the global system to be solved on the finest level L

as $A_L u_L = f_L$, where the restriction of the bilinear form to the space V_ℓ produces the level-specific linear systems $A_\ell u_\ell = f_\ell$.

5.2 The V-Cycle Algorithm

The action of the GMG preconditioner, denoted as P_ℓ^{-1} , on a residual vector r_ℓ at level ℓ is recursively defined by the V-cycle algorithm. Let P_0^{-1} represent the solver applied on the coarsest level ($\ell = 0$). For $\ell > 0$, starting with an initial guess $x^{(0)} = 0$, the preconditioned residual $P_\ell^{-1} r_\ell$ is computed through the following sequence:

1. **Pre-smoothing:** We compute an approximation $x^{(m_\ell)}$ by applying m_ℓ iterations of a smoothing operator S_ℓ :

$$x^{(i)} = x^{(i-1)} + S_\ell^{(i)}(r_\ell - A_\ell x^{(i-1)}), \quad i = 1, \dots, m_\ell \quad (5.1)$$

where S_ℓ acts on the high-energy components of the residual.

2. **Coarse Grid Correction:** The smoothed residual is projected onto the coarser level $\ell - 1$ using the restriction operator $R_{\ell-1}$. The coarse error is approximated by applying $P_{\ell-1}^{-1}$, and the resulting correction is mapped back to the fine level via the prolongation operator $R_{\ell-1}^T$:

$$y^{(0)} = x^{(m_\ell)} + R_{\ell-1}^T P_{\ell-1}^{-1} R_{\ell-1}(r_\ell - A_\ell x^{(m_\ell)}) \quad (5.2)$$

To preserve the symmetry of the preconditioner, the restriction operator $R_{\ell-1} : V_\ell \rightarrow V_{\ell-1}$ is defined exactly as the transpose of the prolongation operator $R_{\ell-1}^T : V_{\ell-1} \rightarrow V_\ell$.

3. **Post-smoothing:** Finally, $y^{(m_\ell)}$ is computed by iteratively applying m_ℓ steps of the smoother to damp any new high-frequency errors introduced by the prolongation process:

$$y^{(i)} = y^{(i-1)} + S_\ell^{(m_\ell+i)}(r_\ell - A_\ell y^{(i-1)}), \quad i = 1, \dots, m_\ell \quad (5.3)$$

We set the output of the preconditioner as $P_\ell^{-1} r_\ell = y^{(m_\ell)}$.

5.3 Chebyshev Polynomial Smoothing

Historically, the Gauss–Seidel method and its variants have been the standard choice for multigrid smoothing due to its excellent damping of high-frequency error components. However, its application in modern parallel and matrix-free architectures presents severe limitations.

Furthermore, Gauss–Seidel is inherently tied to the explicit knowledge of matrix entries, making it fundamentally incompatible with the matrix-free approach. In contrast, polynomial smoothers rely exclusively on global matrix-vector products, requiring no special matrix storage formats.

5.3.1 General Formulation and Symmetric Case

The general polynomial iteration takes the form:

$$x^{(m+1)} = x^{(m)} + p(A)(b - Ax^{(m)}) \quad (5.4)$$

where $p(A) = \sum_{j=0}^n a_j A^j$ is a polynomial of degree n . The error propagation is governed by $e^{(m+1)} = q(A)e^{(m)}$, where $q(A) = I - p(A)A$ is a polynomial of degree $n + 1$ satisfying the constraint $q(0) = 1$.

To act as an ideal smoother, $q(A)$ must complement the coarse grid correction by strongly damping the error associated with high-energy eigenvalues.

For an SPD matrix, we define a cutoff eigenvalue λ_α that separates low and high frequencies. The optimal smoother minimizes the maximum amplitude of $q(\lambda)$ over the interval $[\lambda_\alpha, \lambda_{max}]$. Chebyshev polynomials provide the analytical optimal solution, as they minimize the maximum absolute value of the error polynomial $|q(\lambda)|$ over this specific interval.

The stability of the method is highly sensitive to how the maximum eigenvalue, λ_{max} , is estimated. Since Chebyshev polynomials exhibit rapid, exponential growth outside their optimization domain, any true eigenvalue exceeding the estimated bound will result in algorithmic divergence. Conversely, the method is highly robust against the overestimation of λ_{max} .

5.3.2 The Non-Symmetric Challenge and Complex Ellipses

The classical analysis of the Chebyshev smoother assumes that A is symmetric and positive definite, yielding a strictly real spectrum. However, in the Advection-Diffusion-Reaction equation, the convective term leads to a non-symmetric global matrix A , shifting its eigenvalues into the complex plane. Relying on simple real spectral bounds (e.g., estimating only a 1D real interval $[\lambda_\alpha, \lambda_{max}]$) for a complex spectrum is dangerous because Chebyshev polynomials exhibit rapid exponential growth outside their optimization domain, resulting in immediate algorithmic divergence.

To restore convergence theoretically, the optimization domain must be extended. The application of the Chebyshev iteration to non-symmetric systems is mathematically sound provided that the entire spectrum of A strictly lies in the open right-half complex plane ($\text{Re}(\lambda_i) > 0$). When this condition is met, the optimization boundaries shift from a real segment to a continuous family of confocal ellipses in the complex plane.

The generalized error polynomial, corresponding to $q(\lambda)$ from the symmetric case, is based on scaled and translated Chebyshev polynomials of the form:

$$P_n(\lambda) = \frac{T_n\left(\frac{d-\lambda}{c}\right)}{T_n\left(\frac{d}{c}\right)} \quad (5.5)$$

where $T_n(z)$ is the standard Chebyshev polynomial of degree n in the complex plane. The parameters characterizing the bounding ellipse are its center d and its focal distance c (with foci located at $d + c$ and $d - c$). Since A is a real-valued matrix, its complex eigenvalues appear in conjugate pairs, implying that d and c^2 must be purely real parameters.

While mathematically elegant, tracking complex eigenvalues to determine the optimal parameters d and c adds considerable algorithmic complexity and computational overhead to the multigrid setup phase.

6 | Numerical Results

6.1 Convergence Analysis

Since the matrix-free operator is never assembled as an explicit matrix, its correctness cannot be inspected algebraically and can only be assessed through the convergence of the computed solution. To this end we adopt the **Method of Manufactured Solutions (MMS)**: a smooth analytical solution u_{ex} is prescribed, the corresponding forcing term f and boundary data are derived analytically by substitution into the strong form, and the discrete error is measured against u_{ex} upon mesh refinement.

6.1.1 Homogeneous Dirichlet boundary conditions

The first problem is a full advection-diffusion-reaction problem on the unit cube with homogeneous Dirichlet conditions on the whole boundary,

$$\begin{aligned}\Omega &= [0, 1]^3, & u &= 0 \text{ on } \partial\Omega, \\ \mu &= 0.1, & \boldsymbol{\beta}(\mathbf{x}) &= 0.2 (0.5 - y, x - 0.5, 1)^T, & \gamma &= 0.01, \\ u_{ex}(x, y, z) &= \sin(\pi x) \sin(\pi y) \sin(\pi z).\end{aligned}$$

The advective field is a rotation about the vertical axis superimposed to a vertical drift; it is solenoidal, $\nabla \cdot \boldsymbol{\beta} = 0$, so that $\nabla \cdot (\boldsymbol{\beta} u) = \boldsymbol{\beta} \cdot \nabla u$. The manufactured solution vanishes on $\partial\Omega$, which is consistent with the homogeneous Dirichlet data.

Substituting u_{ex} into the strong form $-\mu\Delta u + \boldsymbol{\beta} \cdot \nabla u + \gamma u = f$ and using $\Delta u_{ex} = -3\pi^2 u_{ex}$, the forcing term is

$$\begin{aligned}f(x, y, z) &= (0.3\pi^2 + 0.01) \sin(\pi x) \sin(\pi y) \sin(\pi z) \\ &\quad + 0.2\pi \left[(0.5 - y) \cos(\pi x) \sin(\pi y) \sin(\pi z) + (x - 0.5) \sin(\pi x) \cos(\pi y) \sin(\pi z) \right. \\ &\quad \left. + \sin(\pi x) \sin(\pi y) \cos(\pi z) \right],\end{aligned}$$

The analysis was conducted using polynomials of degree $p = 2$. The table below summarizes the error norms and their respective convergence rates as the mesh size h decreases:

Cells	DoFs	h	L^2 Error	Rate	H^1 Error	Rate
4096	35937	1/16	2.227e-05	-	2.765e-03	-
32768	274625	1/32	2.787e-06	3.00	6.911e-04	2.00
262144	2146689	1/64	3.485e-07	3.00	1.728e-04	2.00
2097152	16974593	1/128	4.356e-08	3.00	4.319e-05	2.00

Table 6.1: Convergence rates using Q_2 elements

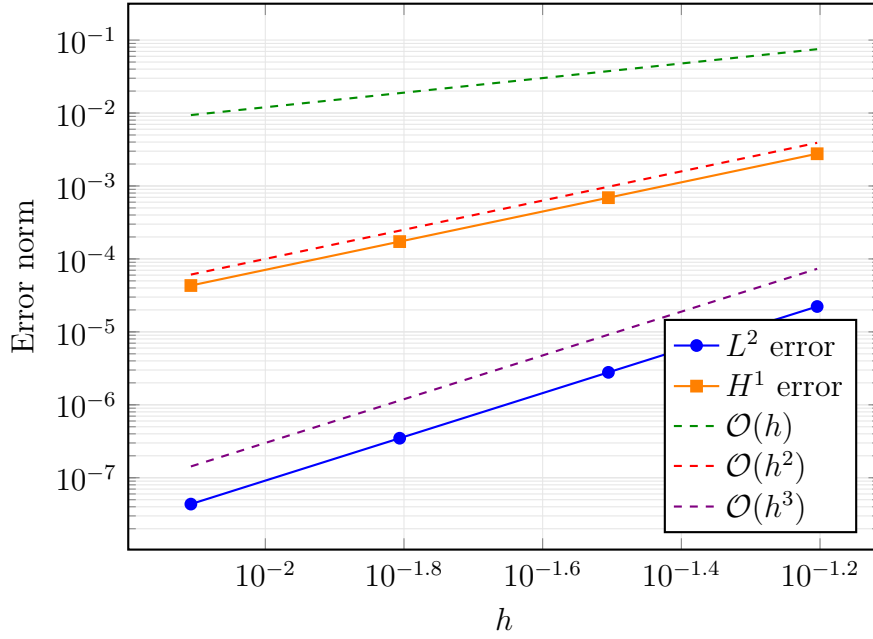


Figure 6.1: Log-log plot of the L^2 and H^1 error norms against the mesh size h using Q_2 elements

6.1.2 Inhomogeneous boundary conditions

The second problem stresses inhomogeneous Dirichlet data, a Neumann boundary, and a spatially varying diffusion coefficient,

$$\Omega = [0, 1]^3, \quad \Gamma_N = \{z = 1\}, \quad \Gamma_D = \partial\Omega \setminus \Gamma_N,$$

$$\mu(\mathbf{x}) = 1 + x, \quad \boldsymbol{\beta} = (1, 0, 0)^T, \quad \gamma = 1, \quad u_{ex}(x, y, z) = x^2y + z^2$$

Enforcing u_{ex} in the strong form $-\nabla \cdot (\mu \nabla u) + \boldsymbol{\beta} \cdot \nabla u + \gamma u = f$, with $\nabla u_{ex} = (2xy, x^2, 2z)^T$ gives

$$f(x, y, z) = x^2y + z^2 - 2xy - 2x - 2y - 2.$$

The Dirichlet datum is $g = u_{ex}|_{\Gamma_D}$, while the Neumann datum is the diffusive flux $\mu \nabla u_{ex} \cdot \mathbf{n}$ on Γ_N ; since $\mathbf{n} = (0, 0, 1)^T$ there,

$$h = \mu \nabla u_{ex} \cdot \mathbf{n} = (1 + x)(2z)|_{z=1} = 2 + 2x$$

Cells	DoFs	h	L^2 Error	Rate	H^1 Error	Rate
4096	4913	1/16	2.963e-03	-	1.082e-01	-
32768	35937	1/32	7.407e-04	2.00	5.412e-02	1.00
262144	274625	1/64	1.852e-04	2.00	2.706e-02	1.00
2097152	2146689	1/128	4.629e-05	2.00	1.353e-02	1.00

Table 6.2: Convergence rates using Q_1 elements

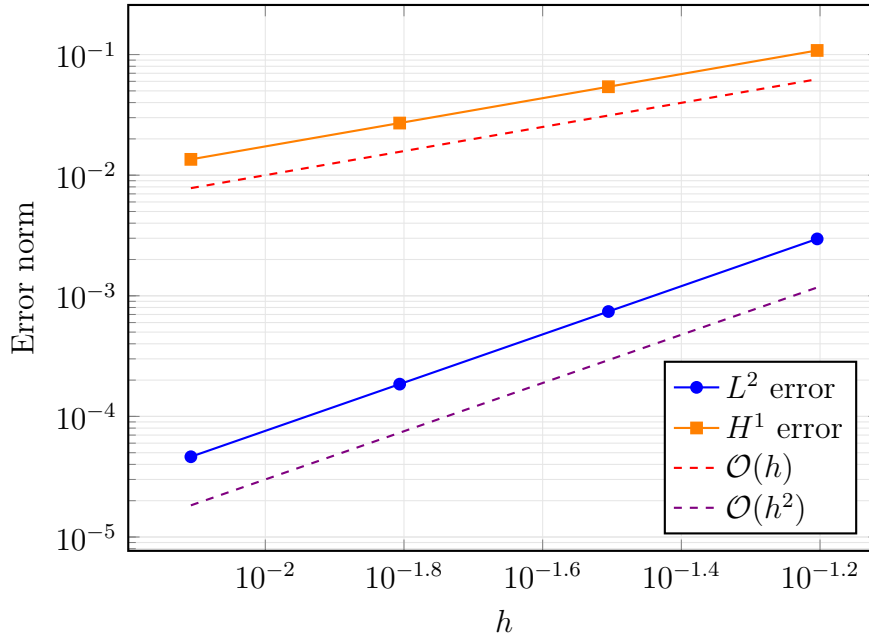


Figure 6.2: Log-log plot of the L^2 and H^1 error norms against the mesh size h using Q_1 elements

The numerical results show perfect agreement with the theoretical error estimates for finite element methods. The expected asymptotic convergence rates are:

- $\mathcal{O}(h^{p+1})$ for the L^2 norm, yielding a theoretical rate of 3 for the first test and 2 for the second one;
- $\mathcal{O}(h^p)$ for the H^1 semi-norm, yielding a theoretical rate of 2 for the first test and 1 for the second one.

This confirms without ambiguity that the matrix-free evaluation of the local integrals, the mapping from reference to real cells, and the application of boundary conditions behave as expected.

6.2 Performance and Scalability Analysis

Beyond the manufactured-solution verification, the solver has been tested on a transport configuration: a localized profile is injected at an inlet face and left to evolve through the domain. The two cases share geometry and data and differ only in the advective field, which isolates the effect of advection and the role of the SUPG stabilization. Both are posed on the unit cube with

$$\Omega = [0, 1]^3, \quad \mu = 0.1, \quad \gamma = 0.01, \quad f = 0,$$

a Gaussian profile prescribed on the inlet face $\Gamma_{\text{in}} = \{z = 0\}$ and homogeneous Dirichlet data on the remaining faces,

$$u = g_{\text{in}}(x, y) = \exp\left(-\frac{(x-0.75)^2 + (y-0.5)^2}{2\sigma^2}\right), \quad \sigma = 0.15 \quad \text{on } \Gamma_{\text{in}}, \quad u = 0 \quad \text{elsewhere.}$$

6.2.1 Pure diffusion–reaction

With no advection ($\beta = \mathbf{0}$) the operator is symmetric: the injected profile spreads isotropically from the inlet while decaying through the reaction term.

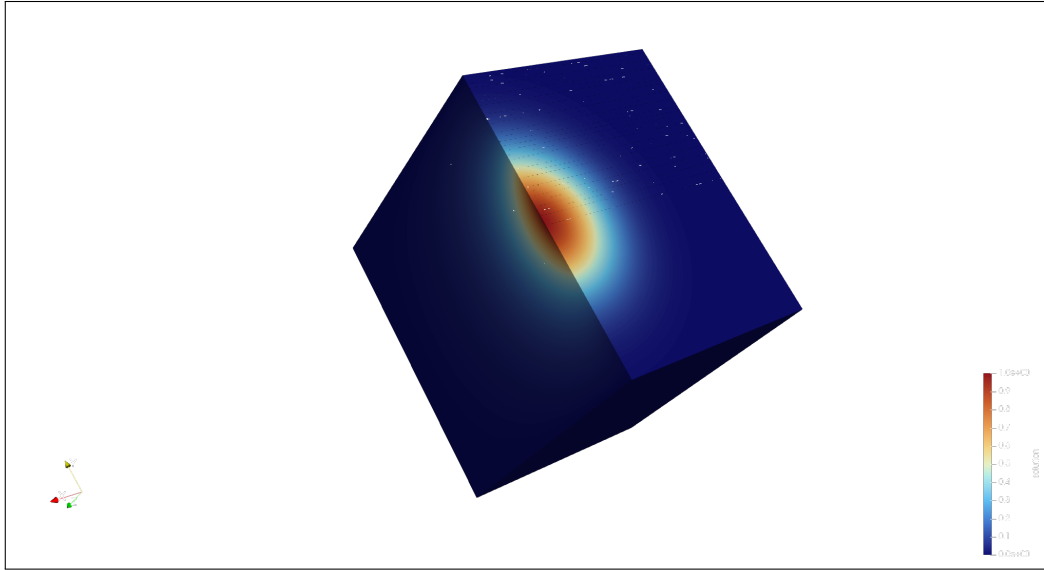


Figure 6.3: Test 1 ($\beta = \mathbf{0}$): pure diffusion–reaction.

6.2.2 Advection-dominated transport (SUPG)

The advective field $\beta = 0.2(0.5 - y, x - 0.5, 1)^T$ transports the profile from the inlet towards the outlet $\{z = 1\}$. This term makes the operator non-symmetric and, in this transport-dominated regime, the standard Galerkin discretization develops spurious oscillations. The SUPG stabilization of Section 3 is therefore switched on, recovering a stable and physically meaningful behavior.

All of the following analysis were carried out on both Test 1 and Test 2; without loss of generality, we have only reported the results related to Test 1. The tests were run on the Department’s student HPC cluster. The benchmarks used the CPU partition, whose nodes are equipped with 112 hardware threads, 0.5 TB of RAM and support AVX-512 vectorization. Each job is subject to a 96 GB memory budget and a limit of 28 cores.

6.3 Strong Scaling

To evaluate the parallel efficiency and absolute performance of the solvers, a strong scaling test was conducted. In this benchmark, the global problem size was kept rigidly fixed at approximately 16.9 million DoFs using quadratic polynomials ($p = 2$), while the number of MPI processes was progressively increased from 1 up to 32. Table 6.3 reports the solve-phase time for both solvers.

MPI Processes	Matrix-Free (s)	Matrix-Based (s)	Speedup
1	80.6	<i>OOM / SegFault</i>	–
2	41.7	748	$\sim 17.9\times$
4	24.7	129	$\sim 5.2\times$
8	13.2	71	$\sim 5.4\times$
16	7.53	42.5	$\sim 5.6\times$
32	4.85	34.8	$\sim 7.2\times$

Table 6.3: Solve-phase time for the Matrix-Free and Matrix-Based solvers ($p = 2$, $\sim 16.9\text{M}$ DoFs).

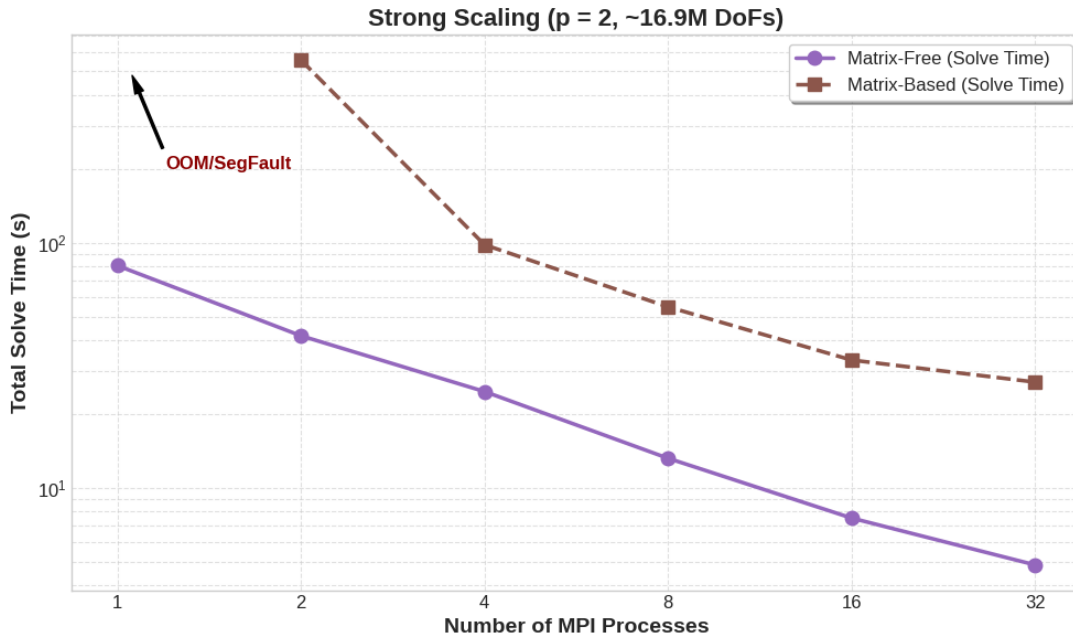


Figure 6.4: Strong scaling ($p = 2$, $\sim 16.9\text{M}$ DoFs).

The performance data highlights a considerable contrast in scalability and hardware utilization between the two approaches. Firstly, the Matrix-Based approach is severely constrained by its memory footprint. It runs out of memory on a single process: storing the global sparse matrix for 16.9M DoFs exceeds the node memory, whereas the matrix-free solver completes the same problem on one core. Moreover, across the entire strong scaling spectrum, the Matrix-Free method demonstrates absolute performance dominance, yielding speedups between 6x and 9x compared to the Matrix-Based. This is because the Matrix-Based solver’s scalability is heavily penalized by the memory bandwidth wall; the necessity to fetch matrix entries from the main memory during both the explicit assembly phase and the Sparse Matrix-Vector (SpMV) multiplications prevents the solver from achieving optimal parallel efficiency. On the other hand, the Matrix-Free solver shows excellent scalability. It can run at its absolute maximum speed, limited only by the processor’s calculation power rather than memory constraints.

6.4 Weak Scaling

To further assess the parallel efficiency of the solvers, a weak scaling test was performed. In this scenario, the workload per processor is kept constant at approximately **274,000 DoFs per MPI process**. As the number of processes increases from 1 to 32, the global problem size proportionally scales up, reaching a maximum of roughly 8.52 million DoFs.

In an ideal weak scaling test, the execution time should remain perfectly constant, indicating that the system handles larger problems without any additional overhead as long as proportional computational resources are provided.

The recorded solve times (in seconds) are detailed in the following table:

MPI Processes	Total DoFs	Matrix-Free (s)	Matrix-Based (s)	Speedup
1	274k	1.03	7.08	~6.9x
2	545k	1.09	8.02	~7.4x
4	1.08M	1.24	8.98	~7.2x
8	2.15M	1.26	10.10	~8.0x
16	4.28M	1.53	11.60	~7.6x
32	8.52M	1.93	17.90	~9.3x

Table 6.4: Solve time comparison for the weak scaling test

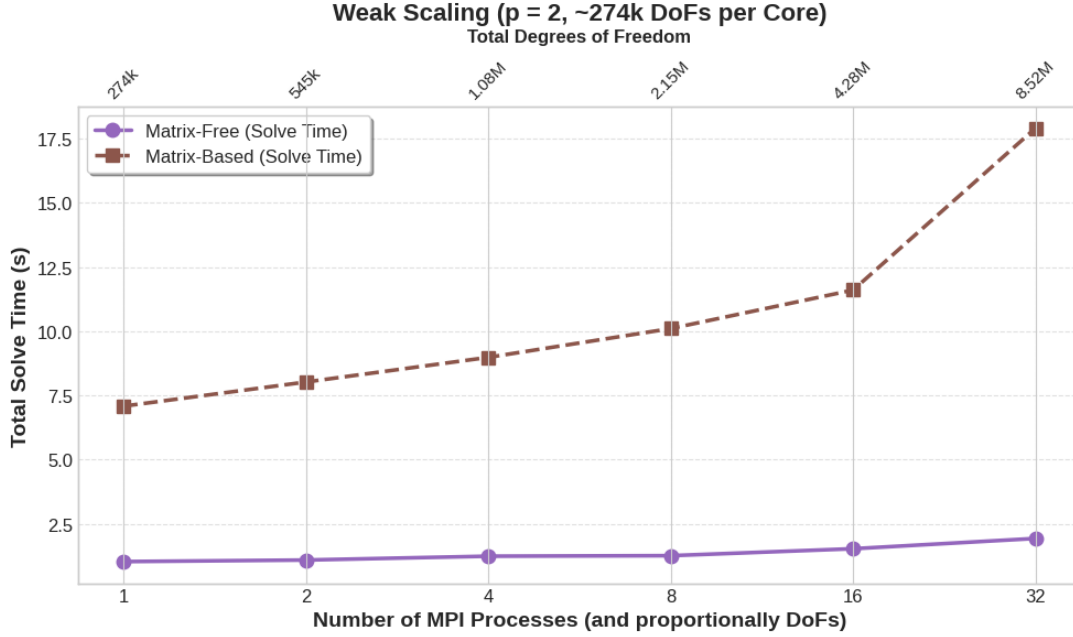


Figure 6.6: Weak Scaling ($p = 2$, $\sim 274\text{k}$ DoFs per Core).

The results clearly illustrate the superior scalability of the Matrix-Free architecture.

The Matrix-Free solver exhibits near-ideal weak scaling characteristics. Its performance curve remains remarkably flat across the entire range, with the solve time only marginally increasing from 1.03 seconds to 1.93 seconds despite the problem size growing by a factor of 32. This slight deviation is completely expected in massive parallel simulations due to the global communication required by the solvers. So, by avoiding the construction of a global sparse matrix, the Matrix-Free method completely bypasses heavy memory communication bottlenecks between nodes.

Conversely, the Matrix-Based solver degrades significantly as the dimension scales. Its execution time more than doubles, jumping from 7.08 seconds at 1 process to 17.90 seconds at 32 processes. This highlights the fundamental limitation of explicitly storing and distributing a global matrix: as the total problem size increases, assembling and communicating the matrix across the MPI boundaries becomes strongly expensive, preventing the system from maintaining constant execution times.

6.5 Polynomial Scaling

To evaluate the algorithmic efficiency with respect to the polynomial degree, a scaling test was performed keeping the hardware configuration fixed at 32 MPI processes. Since varying the polynomial order naturally changes the total number of Degrees of Freedom (DoFs) for a given mesh, evaluating absolute solve times can be misleading. Therefore, the performance is measured in terms of **Throughput**, defined as the number of millions of DoFs processed per second (MDoFs/s). A higher throughput indicates better hardware utilization.

The table below summarizes the Throughput for both the entire resolution process (Full Solve) and a single Multigrid V-Cycle, ranging from linear to high-order polynomials:

Degree	FULL Solve Throughput (MDoFs/s)		V-Cycle Throughput (MDoFs/s)	
	Matrix-Free	Matrix-Based	Matrix-Free	Matrix-Based
1	19.52	23.65	7.56	51.11
2	26.41	13.16	10.22	24.67
3	27.54	9.92	10.40	12.93
4	49.61	7.20	18.35	11.12
5	30.40	3.23	9.89	4.59
6	45.98	3.62	15.30	4.82
7	37.61	2.56	14.43	3.75

Table 6.5: Throughput comparison between Matrix-Free and Matrix-Based solvers across different polynomial degrees.

Polynomial Scaling Results (32 MPI Processes, ~2M-7M DoFs)								
Degree (p)	Solver	DoFs	Iterations	V-cycle (s)	Solve (s)	Ass+Setup (s)	Throughput_Solve (MDoF/s)	Throughput_Vcycle (MDoF/s)
1	MF	2146689	10	0.284	1.1	1.118	19.52	7.56
1	MB	2146689	13	0.042	1.18	2.09	23.65	51.11
2	MF	2146689	11	0.21	0.894	0.615	26.41	10.22
2	MB	2146689	32	0.087	5.22	3.97	13.16	24.67
3	MF	7189057	10	0.691	2.61	1.575	27.54	10.4
3	MB	7189057	45	0.556	32.6	36.97	9.92	12.93
4	MF	2146689	11	0.117	0.476	0.272	49.61	18.35
4	MB	2146689	49	0.193	14.6	28.69	7.2	11.12
5	MF	4173281	11	0.422	1.51	1.327	30.4	9.89
5	MB	4173281	52	0.91	67.1	166.52	3.23	4.59
6	MF	7189057	11	0.47	1.72	1.186	45.98	15.3
6	MB	7189057	63	1.49	125.0	454.2	3.62	4.82
7	MF	1442897	11	0.1	0.422	0.489	37.61	14.43
7	MB	1442897	67	0.385	37.8	358.17	2.56	3.75

Figure 6.8: Performance breakdown for the polynomial scaling test.

The experimental throughput data confirms that the Matrix-Free solver scales significantly better than the Matrix-Based solver as the polynomial degree increases. Theoretically, the local computational complexity per cell of the Matrix-Based solver scales as $\mathcal{O}(p^{2d})$, which corresponds to $\mathcal{O}(p^6)$ in 3D ($d = 3$). On the other hand, the Matrix-Free solver with sum-factorization reduces the local complexity to $\mathcal{O}(d \cdot p^{d+1})$ ($\mathcal{O}(p^4)$ in 3D).

This difference in theoretical complexity is clearly visible in the throughput results. While the Matrix-Based solver is faster for linear elements ($p = 1$) due to the low coupling and highly optimized sparse matrix-vector operations, the Matrix-Free solver becomes significantly more efficient for high-order polynomials ($p \geq 2$).

6.6 Memory Profiling

The defining advantage of the matrix-free approach is not only speed but memory footprint: by never assembling the global sparse matrix, the solver stores only the solution vectors and a compact set of per-cell geometric data. The test was executed by using quadratic elements ($p = 2$) on 32 MPI processes, while increasing the problem size from roughly 4×10^4 to 1.3×10^8 degrees of freedom. The reported quantity is the Resident Set Size (RSS) aggregated over all 32 processes which sums the per-process VmRSS via an MPI reduction. The results are shown in Figure 6.9.

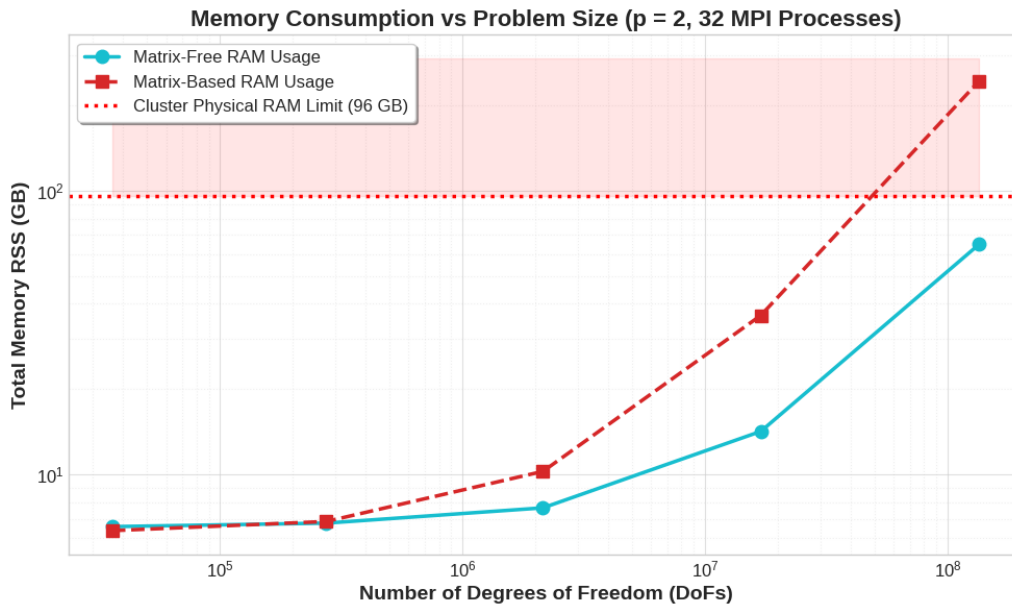


Figure 6.9: Total memory consumption (aggregated RSS over 32 MPI processes) versus problem size, for $p = 2$.

For small problems (up to about 3×10^5 DoFs) the two curves are nearly indistinguishable and sit around 6–7 GB. In this regime the memory is dominated not by the problem data but by fixed overheads. So the storage of the matrix is negligible and the two approaches are effectively identical.

The practical consequence is most evident at the largest problem size. At approximately 1.3×10^8 DoFs the Matrix-Based solver reaches about 250 GB, far beyond the 96 GB of physical memory available on the cluster. The value exceeds the physical RAM limit because, once the memory is saturated, part of the data is offloaded to the hard disk; the run does not complete and ultimately fails, consistently with the strong scaling experiment of Section 6.3. The Matrix-Free solver, by contrast, completes the same problem using roughly 65–70 GB, comfortably below the physical limit.

This experiment confirms the central claim of the project from the memory standpoint: the matrix-free operator does not merely run faster, it makes problems *feasible* that the matrix-based approach simply cannot fit into memory on the same hardware.

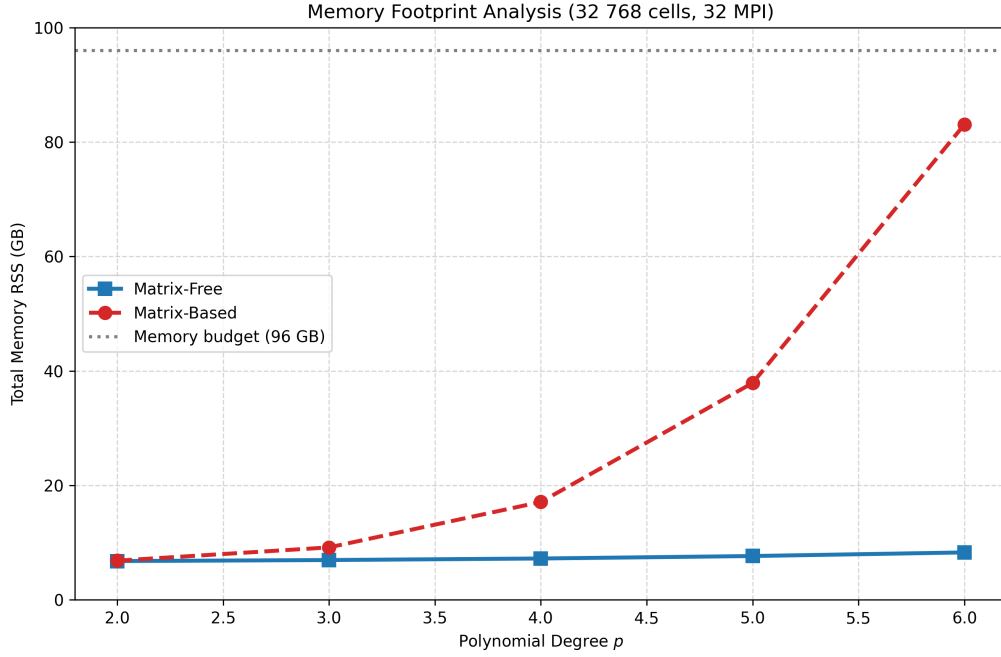


Figure 6.10: Total memory consumption (aggregated RSS over 32 MPI processes) versus problem size, for $p = 2$. The dotted line marks the physical RAM limit of the cluster (96 GB).

A complementary experiment isolates the effect of the polynomial degree: the mesh is kept fixed at 32 768 cells on 32 MPI processes, while p is increased from 2 to 6 (Figure 6.10). On a fixed mesh the two solvers respond very differently. The Matrix-Free footprint stays almost flat, just below 10 GB across the whole range: its storage scales only with the number of DoFs and, at this size, remains dominated by the fixed overhead discussed above. The Matrix-Based footprint, instead, grows steeply an overall $\mathcal{O}(p^{2d})$ growth. This mirrors, on the memory side, the architectural divide already seen in the throughput results 6.5: the explicit matrix scales as $\mathcal{O}(p^{2d})$, whereas the matrix-free operator stays close to linear in the DoFs, so at high order the stored matrix becomes the binding memory constraint while the matrix-free solver is essentially unaffected.

7 | Conclusions

The project successfully demonstrated the advantages of matrix-free solvers for high-order ADR problems. By utilizing sum-factorization and SUPG stabilization, we achieved a solver that is both numerically robust in transport-dominated regimes and computationally superior on modern parallel architectures.

8 | Bibliography

K. Kormann and M. Kronbichler, *The deal.II Tutorial — Step-37: Matrix-free Methods*, deal.II finite element library (version 9.3.1), 2012.

A. Quarteroni, *Numerical Models for Differential Problems*, 3rd ed., MS&A — Modeling, Simulation and Applications, vol. 16, Springer, Cham, 2017.

M. Adams, M. Brezina, J. Hu, and R. Tuminaro, *Parallel multigrid smoothing: polynomial versus Gauss–Seidel*, Journal of Computational Physics, vol. 188, no. 2, pp. 593–610, 2003.

T. A. Manteuffel, *The Tchebychev iteration for nonsymmetric linear systems*, Numerische Mathematik, vol. 28, pp. 307–327, 1977.